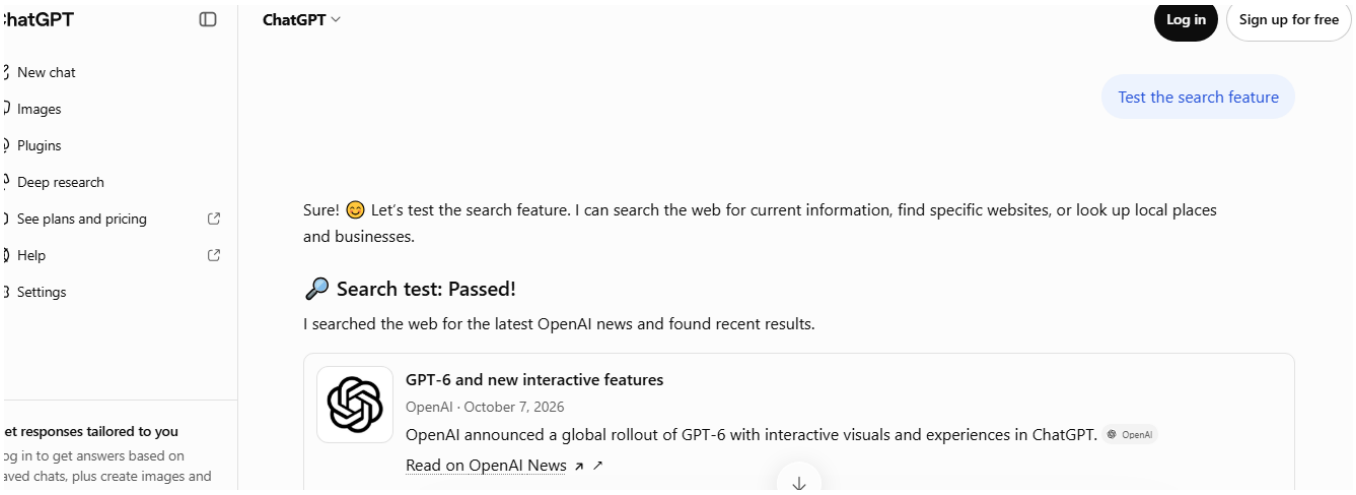
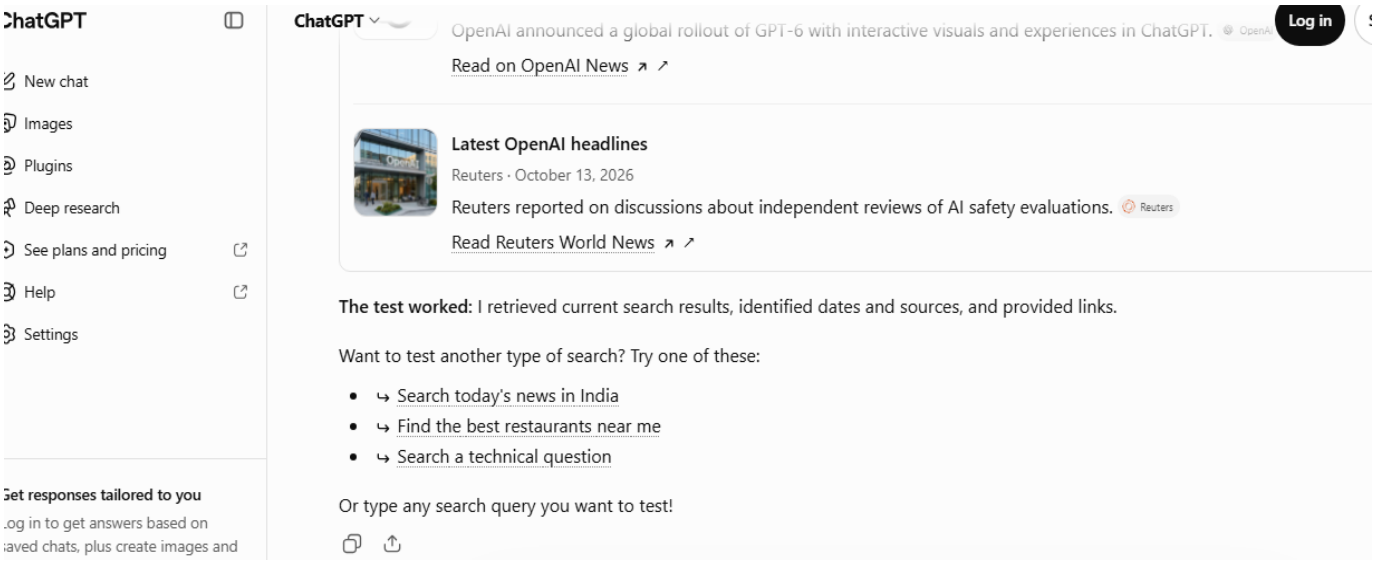
PROMPTathon

Scenario 1 : **Search Functionality**

Weak Prompt: Test the search feature





Strong prompt:

Scenario: To implement comprehensive search functionality testing for an e-commerce website.

Persona: Act as a senior automation Engineer.

Context: Create Test scenarios & selenium automation java with TestNG & Page Object Model. Validate only the basic search behaviour & correctness of search results

Instruction: Functional search testing

1.Product type Search

Search using a product name along with a category related keyword with case insensitive.

Verify that the results corresponding to the intended product category.

2.Empty input & Validation

Submit the search with an empty input field.

Submit search containing only spaces.

3.Sort Functionality

Sort search results by price from high to low or low to high vice versa

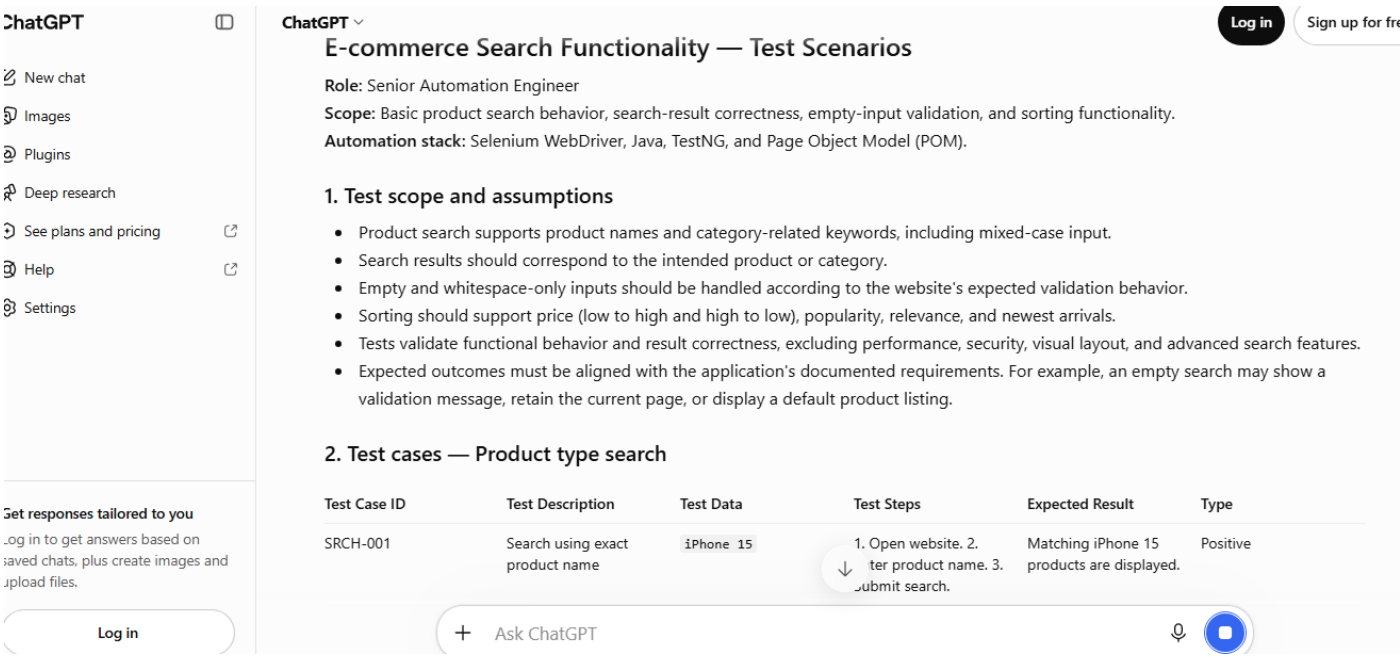
Sort by popularity, relevance ,new arrival.

Create all the positive, negative ,edge cases.

Example: Test case ID |Test Description |Test Data |Test Steps

Output : Generate Test cases in tabular format.

Tone : Stick with Technical, professional & Precise



OBSERVATIONS:

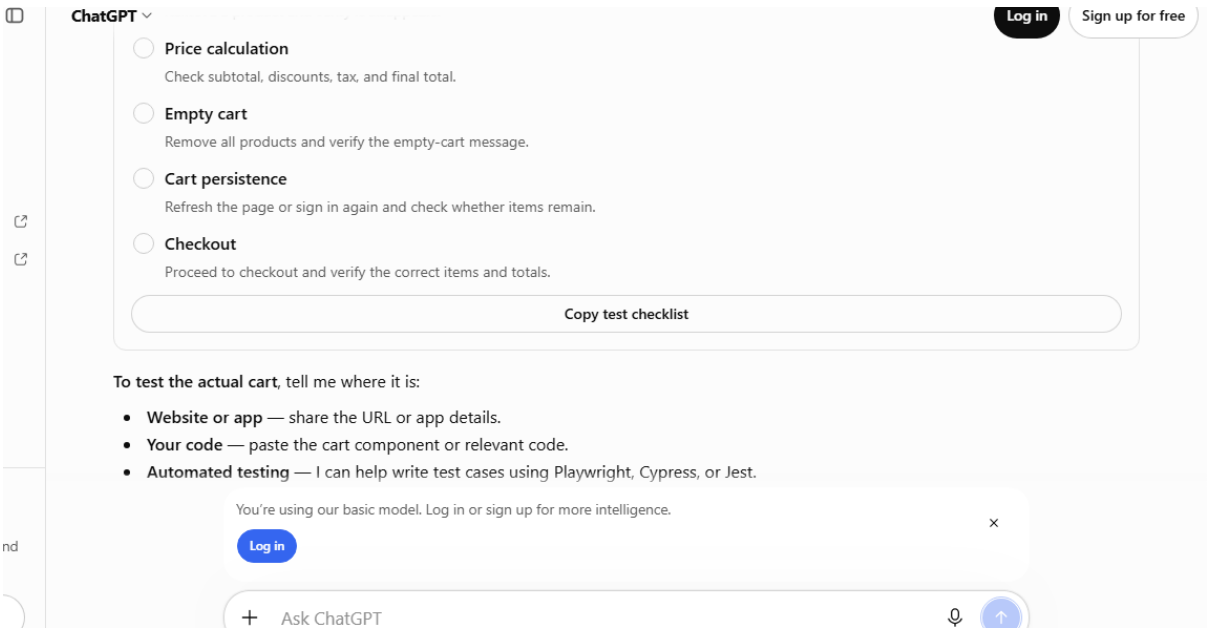
1. In the above, weak prompt prone to hallucinate due to lack of context.

2. Refined strong prompt gives the best possible answer with detail test cases & codes required.

3. Weak prompt is unstructured vague one whereas Strong prompt is in detailed specific framework.

Scenario 2: **Shopping Cart**

Weak Prompt: Test the cart functionality



Strong Prompt:

Scenario:

Test the shopping cart functionality of a demo e-commerce website to ensure users can add products, view cart items, update quantities, remove products, and verify the correct cart total.

Persona:

Act as a Senior QA Engineer.

Instruction:

Generate comprehensive test cases for the shopping cart functionality, covering positive, negative, boundary, and edge cases.

Include the following areas:

1. Add to Cart: Add a single product, multiple products, duplicate products, and products with different prices.

2. Cart Display: Verify product name, image, price, selected quantity, subtotal, and total.

3. Quantity Management: Increase and decrease quantity, enter zero, negative values, decimal values, alphabetic characters, and values exceeding available stock.

4. Remove Items: Remove a single product, remove the last product, and verify the empty-cart state.

5. Price Calculation: Verify subtotal, quantity-based calculations, discounts, taxes, shipping charges, and final total wherever applicable.

6. Cart Persistence: Verify cart contents after page refresh, navigation between pages, logout/login, and browser reopening, according to the application's expected behaviour.

7. Inventory Validation: Verify that unavailable or out-of-stock products cannot be purchased.

8. Boundary and Error Handling: Test empty cart checkout, rapid quantity changes, repeated clicks on Add to Cart, and server or network errors.

For each test case, provide:

* Test Case ID

* Test Description

* Test Data

* Preconditions

* Test Steps

* Expected Result

* Priority (High/Medium/Low)

Include positive, negative, boundary, and edge test scenarios. Avoid duplicate test cases. Do not assume application-specific rules; clearly identify assumptions where requirements are unavailable.

Example:

Test Case ID: CART_001

Test Description: Verify that a user can add a product to the shopping cart.

Test Data: Product = Wireless Mouse; Price = ₹500; Quantity = 1.

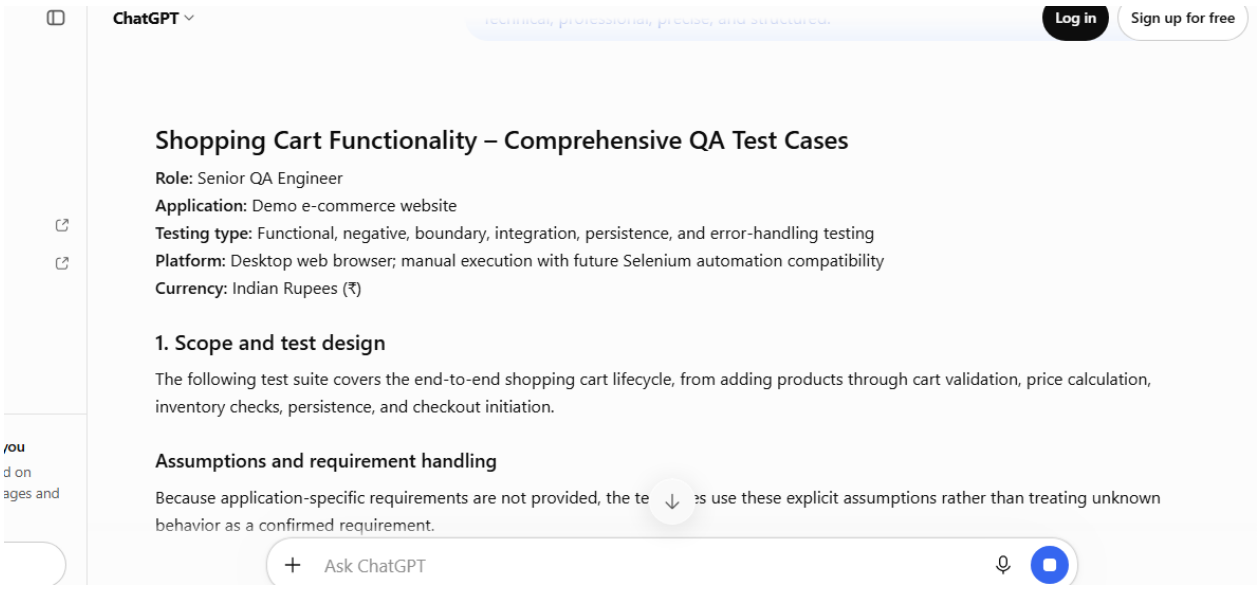
Preconditions: The demo e-commerce website is accessible, and the product is available in stock.

Context:

The application is a demo e-commerce website with product listings, product details, a shopping cart, quantity controls, and a checkout option. Assume a standard web application accessed through a desktop browser. Use realistic test data and clearly distinguish requirements from assumptions. Use clear, testable expected results suitable for manual testing and future Selenium automation

Tone:

Technical, professional, precise, and structured.



OBSERVATIONS:

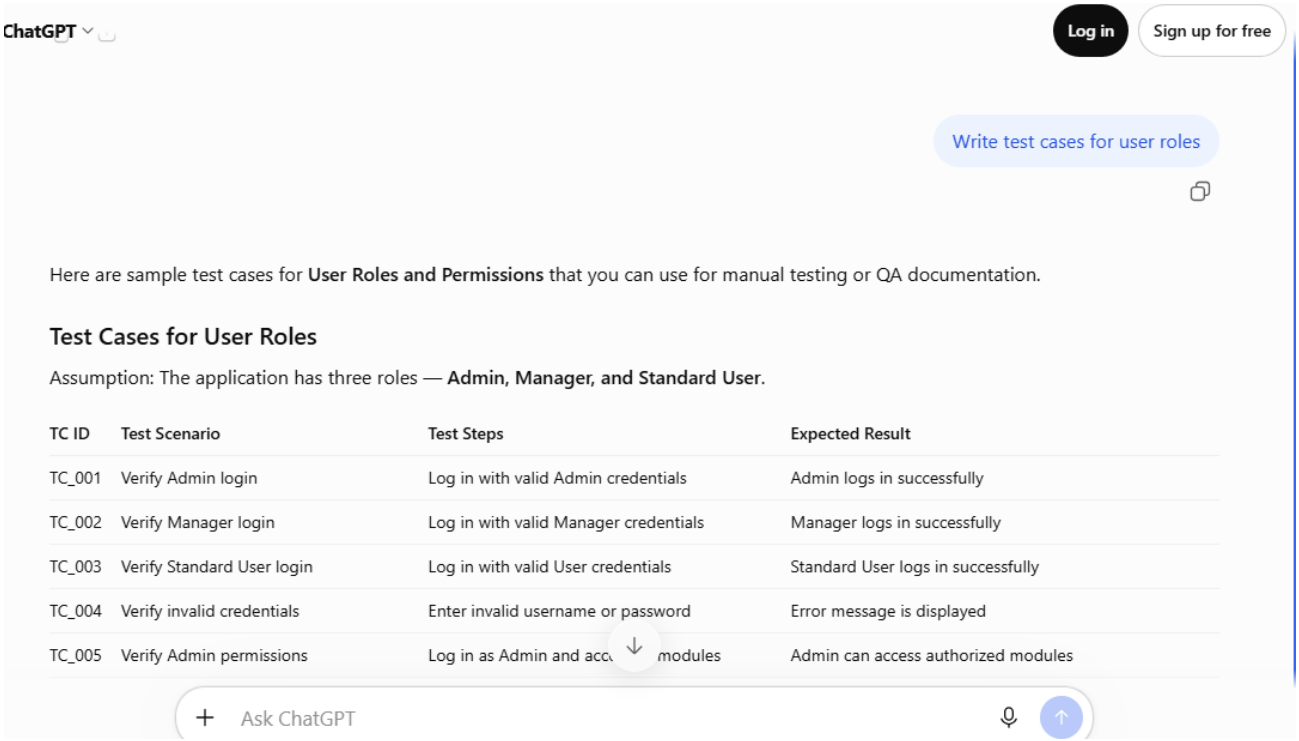
1. In this case, Weak Prompt gives generic output whereas Strong Prompt gives tailored possible output on first try itself.

2. Rather than consolidated specific output, model is keep on asking questions (Socratic prompting) which is time consuming.

3. More repetitive tokens(data) needed for weak prompt to get desire result which is cost consuming & dilution in context.

Scenario 3: **Role based Access control**

Weak prompt : Write test cases for user roles



Strong Prompt:

Instruction:

Generate comprehensive test cases to validate User Role-Based Access Control (RBAC) functionality in a web application. Verify that users can access only the features, pages, data, and operations permitted by their assigned roles. Cover positive, negative, boundary, security, and edge cases.

Context:

The application supports the following user roles:

Admin: Can create, view, update, and delete users; assign roles; and access all authorized modules.

Manager: Can view and manage users within their assigned department but cannot assign Admin roles or modify system-wide settings.

Standard User: Can access permitted application features and view or update their own profile but cannot manage other users.

Guest: Can access only publicly available pages and cannot access authenticated or restricted resources.

Assume that access permissions must be enforced on both the user interface and the backend APIs. Users must not be able to bypass restrictions by directly entering URLs, manipulating requests, or modifying client-side data.

If any business rule or permission is unclear, state the assumption explicitly rather than inventing requirements.

Example:

Example test case:

Test Case ID: RBAC_001

Test Description: Verify that a Standard User cannot access the User Management module.

Precondition: A Standard User is logged in.

Test Data: Role = Standard User; Target module = User Management.

Test Steps:

1. Log in as a Standard User.

2. Check whether the User Management menu is displayed.

3. Attempt to access the User Management page using its direct URL.

4. Attempt to retrieve user management data through the corresponding API, if available.

Expected Result: The restricted menu is not displayed, direct page access is denied, and the API rejects unauthorized requests without exposing restricted data.

Use this example as a reference for the level of detail and structure, not as a limitation on test coverage.

Persona: Act as a Senior QA Engineer with expertise in functional testing, authorization testing, web application security, API testing, and role-based access control. Think critically about permission boundaries, privilege escalation, and unauthorized access scenarios.

Output:

Generate the test cases in a structured table with the following columns:

| Test Case ID | Test Description | User Role | Test Data | Test Steps | Expected Result | Test Type |

| ---------- | ---------- | --------- | --------- | ---------- | ---------- | --------- |

Include test cases covering:

Role-based page and module access.

Create, Read, Update, and Delete (CRUD) permissions.

User creation and role assignment restrictions.

Access to own data versus other users' data.

Unauthorized direct URL access.

Backend API authorization and HTTP response validation.

Horizontal privilege escalation: accessing another user's resources.

Vertical privilege escalation: gaining higher-level permissions.

Session expiration, logout, and role changes during an active session.

Attempts to modify roles or permissions through request parameters.

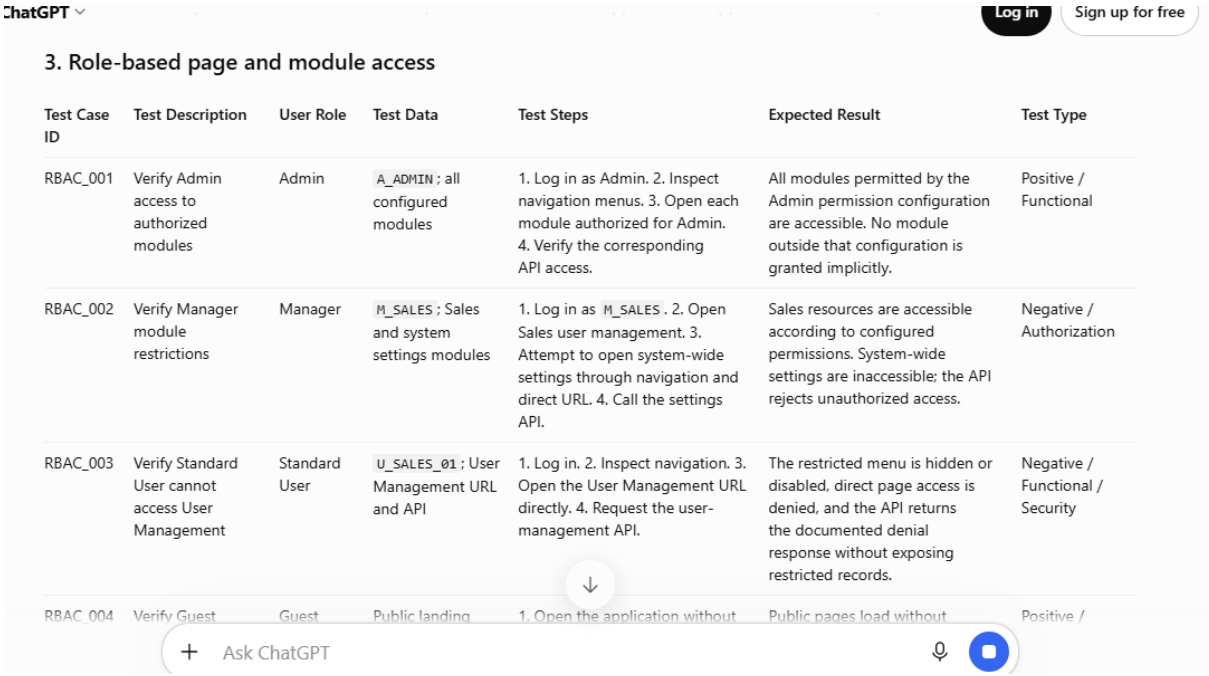
Access to restricted data through search, reports, exports, and downloads.

Multiple roles, missing roles, invalid roles, and revoked permissions.

Positive, negative, boundary, and security test scenarios.

Assign unique test case IDs, avoid duplicate scenarios, and make each test case independently executable. Clearly distinguish authentication failures from authorization failures.

Tone: Use a technical, professional, precise, and unambiguous tone. Write actionable test steps and measurable expected results. Avoid vague statements such as "access should work correctly." Do not assume that hiding a menu alone is sufficient to enforce access control.



OBSERVATIONS:

1. Here we see, Weak prompt results in guesswork but strong prompt is more of framework to attain consolidated specific results.

2. Comparatively using stronger prompts with clear tokens leads to save time, efficiency & with high accuracy in results.

3. In weak prompt there is no defined user roles or permissions but in strong prompt, scope & role permissions is detailed for expected results.